

$c \rightarrow D^0$ fragmentation functions

October 4, 2026

The code has three FF: BCFY, KniehlKramer and HymnD.

1 Variables

The fragmentation fraction z_h is the fraction of the charm quark transverse momentum carried by the D^0 ,

$$p_{D^0} = z_h p_c, \quad (1)$$

where p_{D^0} is `pD0` in the code and p_c is `pc`. The integrand is multiplied by

$$\frac{D(z_h, m_T)}{z_h^2}, \quad z_h \in [z_{h,\min}, z_{h,\max}] = [0.05, 1], \quad (2)$$

and z_h is one of the variables of the Monte Carlo integration (VEGAS). The limits are `z_h_min` and `z_h_max` in `src/main.cpp`.

The FF is taken at the fragmentation scale

$$m_T = \kappa \sqrt{p_{D^0}^2 + m_c^2}, \quad \kappa = \text{SCALE_FACTOR (default 1)}, \quad (3)$$

where m_c is the charm quark mass. If the scale is below m_c the code uses $m_T = m_c$.

2 How the FF gets into the calculation

The three FFs are tables in the LHAPDF format (`lhagrid1`), on a grid in z_h and m_T . They are read by the same function, `MakeHymnDZInterpolator(file, flavor, m_T)` in `src/hymnd_grid.cpp`:

1. it takes the column of the charm quark (PDG id 4),
2. it interpolates linearly in m_T between the two closest grid values. If m_T is outside the grid, it uses the closest grid value,
3. the table has $z_h D(z_h, m_T)$, so it divides by z_h ,
4. it gives back an `Interpolator` for $D(z_h)$ at this m_T .

This is done once per run in `src/main.cpp`, because m_T depends only on p_{D^0} . The result is stored in `param.D_frag_interp`. In `src/cross_section_inclusive.cpp` the integrand uses it in one place:

```
D_frag = par->D_frag_interp->Evaluate(z_h);  
double frag_weight = D_frag / (z_h * z_h);
```

Then, the FF can be changed without modifying the integrand.

3 The fragmentation functions in this code

3.1 BCFY (Braaten–Cheung–Fleming–Yuan)

File `input/BCFY_EKO/bcfy_eko_0000.dat`. The initial value at $m_T = m_c$ is the BCFY analytical function and it is evolved to higher scales with DGLAP (EKO, in the repository EKO-FF). The grid has 200 points in z_h from 0.05 to 1 and 30 points in m_T , starting at m_c .

At this scale it is the sum of the part for the D^0 produced directly and the part for $D^{*0} \rightarrow D^0$, with the formulae of Braaten, Cheung, Fleming and Yuan, Phys. Rev. D **51** (1995) 4819 (arXiv:hep-ph/9409316). The parameter is $r = 0.1$ and the weights of the two parts are 0.168 and 0.39.

3.2 Kniehl–Kramer

File `input/KK_EKO/kk_eko_0000.dat`, with the same grid and the same evolution as BCFY. At $m_T = m_c$ it is the function of Kniehl and Kramer, Phys. Rev. D **74** (2006) 037502 (arXiv:hep-ph/0607306),

$$D(z_h) = \frac{N z_h (1 - z_h)^2}{[(1 - z_h)^2 + \epsilon z_h]^2}, \quad N = 0.694, \quad \epsilon = 0.101. \quad (4)$$

3.3 HymnD

File `input/HymnD/prompt-D0-1-109_0000.dat`, the central member of the set of Epele, Hekhorn, Helenius, Paukkunen and Zurita, “Towards new D meson fragmentation functions” (arXiv:2609.10327). The set has 100 more members (replicas), which give the uncertainty band of the FF: `run_scripts/run_members.sh` with `MEMBER_SET=HymnD` runs the calculation once per member. All members are in `input/HymnD/` as `prompt-D0-1-109_NNNN.dat`.

4 How to choose the FF

The FF is chosen with environment variables (the defaults are in `run_scripts/config.sh`):

Variable	Default	
FRAG_TYPE	KniehlKramer	FF: BCFY, KniehlKramer or HymnD
SCALE_FACTOR	1.0	κ of the fragmentation scale
SCALE_FACTORS	0.5 1.0 2.0	values of κ in <code>run_scale_variation.sh</code>
HYMND_FILE	central member	file of HymnD
BCFY_EKO_FILE	<code>input/BCFY_EKO/bcfy_eko_0000.dat</code>	file of BCFY
KK_EKO_FILE	<code>input/KK_EKO/kk_eko_0000.dat</code>	file of KniehlKramer

For example:

```
FRAG_TYPE=BCFY ./build/bin/dipole 3 data/Pb/mve/glauber_mve_0 0
CHANNEL=Xn0n FRAG_TYPE=BCFY ./run_scripts/run_nucleus.sh
FRAG_TYPE=HymnD ./run_scripts/run_scale_variation.sh
```

The results go to a folder with the name of the FF, `output/<CHANNEL>/central_values/<FRAG>/` and `output/<CHANNEL>/scale_variation/<FRAG>/factor_< $\kappa$ >/`.

5 Uncertainty bands

- Scale: the script `run_scale_variation.sh` runs the calculation for each κ in `SCALE_FACTORS`. The plot of `plotting_scripts/cms_comparison.py` uses $\kappa = 0.5, 2$ for HymnD and $\kappa = 0.25, 4$ for Kniehl–Kramer. For HymnD, $\kappa = 0.25$ is not used because $0.25 m_T$ is below the D^0 mass at low p_{D^0} .
- HymnD replicas: run `run_scripts/run_members.sh` with `MEMBER_SET=HymnD`. The results are in `output/<CHANNEL>/HymnD_band/member_NNNN/`.

6 How to add another FF

6.1 A table in the LHAPDF format

Example:

```
HYMND_FILE=path/to/my_ff_0000.dat FRAG_TYPE=HymnD ./run_scripts/run_nucleus.sh
```

The output files then have the name HymnD.

To add it with its own name:

1. Add the name to enum `class FragmentationType` in `src/def.hpp`.

2. In `src/main.cpp`, add the name where `FRAG_TYPE` is read, a line where the FF is built,

```
param.D_frag_interp = MakeHymnDZInterpolator("input/MyFF/my_ff_0000.dat",
                                             4, frag_scale);
```

and a line in the switch that prints # fragmentation : <name>.

3. In `run_scripts/run_nucleus.sh`, add the name where `frag_tag` is read from that line.
If you want it in the plot of `plotting_scripts/cms_comparison.py`, add it to the list of fragmentation functions there too.

The file must have this form (the lines before the first `--` are not read):

```
---
z_h values (one line)
m_T values in GeV (one line)
flavors (one line, it must contain 4)
z_h*D for every flavor, one line per (z_h, m_T): first all m_T for the first z_h, ...
---
```

6.2 A formula

Write a function like `MakeKniehlKramerInterpolator()` that fills two vectors and gives back the interpolator:

```
std::vector<double> z_h_vals(n), dvals(n);
for (int i = 0; i < n; i++) {
    z_h_vals[i] = 0.05 + (1.0 - 0.05) * i / (n - 1.0);
    dvals[i]     = my_D(z_h_vals[i]);
}
return std::unique_ptr<Interpolator>(new Interpolator(z_h_vals, dvals));
```

and then follow the three steps above.

Careful

- The interpolator must give $D(z_h)$. LHAPDF tables have $z_h D$, and `MakeHymnDZInterpolator()` already divides by z_h .
- The grid must cover z_h from 0.05 to 1 and m_T from m_c to the largest scale of the run (the largest κ and p_{D^0}). Below the grid the code uses the lowest value of m_T , above it the highest.